

Lab exercises

Part 14 —  AI Compliance · 14-ai-compliance/lab/EXERCISES.md

Lab — AI Compliance

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. As always in this module: **nothing here is legal advice**, including your own answers — the point is to practice the reasoning, not to generate something you'd hand to a regulator unreviewed.

1. Warm-up: run the tools you were given (30 min)

Run both scripts in `../code/`:

```
python ../code/eu_ai_act_risk_classifier.py --demo
python ../code/compliance_documentation_mapper.py
```

- **1a.** In the classifier's output, name the one input field that flips `Customer-service chatbot (with disclosure)` from a compliant Limited-risk result to a Limited-risk result with a GAP. Change it and re-run to confirm.
- **1b.** In the mapper's output, find a requirement with **zero** full-coverage artifacts. What new artifact (not one of the four) would you need to build to close it?
- **1c.** The classifier refuses to construct a `UseCase` with `claims_narrow_task_exception=True` and an empty `narrow_task_justification`. Read `__post_init__` and explain in one sentence why that's a deliberate design choice, not an oversight.

Deliverable: three short written answers.

2. Classify five use cases from your own work (or realistic ones) (2h)

Write a JSON file (`my_use_cases.json`) with 5 use cases shaped like `UseCase`'s fields (see the demo list in the script for the exact shape) drawn from your own job, a past project, or plausible ones if you don't have real ones handy. Deliberately include:

- one you're confident is minimal-risk,
- one you're confident is high-risk,
- one you're genuinely unsure about.

Run:

```
python ../code/eu_ai_act_risk_classifier.py --input my_use_cases.json
```

Checks: - Your two confident cases land where you expected. If they don't, that's not a script bug by default — read the reasoning trace and figure out which fact you mis-modeled. - For the uncertain one, write two sentences: what the tool said, and what specific question you would ask a lawyer to resolve the uncertainty (not "is this compliant" — a falsifiable question, per notes/03 §3).

Deliverable: the JSON file, the tool's output, and your two-sentence note.

3. Break the classifier honestly (1h)

Every rule-based classifier has edge cases where the rules disagree with reality. Find (or construct) three use cases where you believe the tool's simplified logic gives an answer a real lawyer would push back on. For each:

- **3a.** State the use case and the tool's output.
- **3b.** State what a more careful reading of the Act would actually say, and why (cite the specific mechanism — an Article 6(3) narrow-task argument, an Annex III boundary case, a jurisdiction question the tool doesn't model at all).
- **3c.** Propose one concrete field or rule you'd add to the classifier to catch this case, without making the tool so complex it stops being legible.

Deliverable: a 3-row table (use case | tool's answer | why a lawyer might disagree | proposed fix). This exercise is the point of the whole module: knowing exactly where your tooling's simplifications live is more valuable than trusting the tool.

4. Extend the documentation mapper (1.5h)

The mapper currently models four artifacts. Add a fifth: **vendor due-diligence record** (the questionnaire/contract review you'd do before adopting a third-party model API or dataset vendor).

- Add it to `ARTIFACTS`.
- Go through every `Requirement` in `REQUIREMENTS` and decide whether your new artifact contributes `full`, `partial`, or `none` — you'll need to actually think about each one, not just mark everything `partial` to be safe.
- Re-run `--json` and confirm the new column appears and the leverage summary picks it up.

Checks: - At least 2 requirements move from "thin coverage" to properly covered because of the new artifact. - You can explain, for one requirement, *why* your new artifact only gets `partial` rather than `full`.

Deliverable: your diff and one paragraph justifying your full/partial/none calls for the ISO 42001 A.9 (third-party/supplier) requirement specifically.

5. The gap-assessment table, for real (2h)

Using the table shape from notes/03 §1 [14-ai-compliance/notes/03-building-a-compliance-program.md], build a gap assessment for a **real product** (your employer's, a side project, or a well-documented public example like a company's published AI feature). Columns: use case, jurisdiction(s), sector rules, AI-specific tier/rule, data-protection posture, owner, status.

- Run the use case through `eu_ai_act_risk_classifier.py` to get the AI-Act column with reasoning, not a guess.
- Run the relevant artifacts you have (or would have) through `compliance_documentation_mapper.py`'s logic to fill in "what's already covered" for the status column.

Deliverable: a one-page (one screen) gap assessment table plus a "what I'd ask counsel to confirm" list of 3 specific, falsifiable questions.

6. Timeline currency check (1h) — do this even if you skip everything else

Go to artificialintelligenceact.eu/implementation-timeline (or the [EU AI Act Service Desk](#)) right now, on the date you're doing this exercise.

- **6a.** Compare what you find against the timeline table in notes/01 [14-ai-compliance/notes/01-regulatory-landscape.md]. List every date that has changed since this module's last verification date (stated at the top of notes/01).
- **6b.** Do the same for one US state law of your choice (Colorado is the running example in notes/01 — pick a different one).
- **6c.** Write one sentence on what this exercise proves about treating any compliance module, including this one, as a permanently accurate source.

Deliverable: your findings from 6a/6b (even if the answer is "nothing changed"), and your 6c sentence. This is the single most important exercise in the lab — a stale compliance program is worse than no compliance program, because it creates false confidence.

7. Stretch: the capstone — present the gap assessment to "legal" (2h)

Pair with someone else (or role-play both sides yourself, in writing). Present exercise 5's gap assessment as if to counsel, following the "what you bring / what you expect back" split in notes/03 §3 [14-ai-compliance/notes/03-building-a-compliance-program.md]. Have the "counsel" side push back with at least two hard questions your gap assessment doesn't currently answer.

Check: you can revise the gap assessment to close both gaps, or explicitly mark them as "needs counsel input, not resolvable from the engineering side" — and can say which of those two outcomes each one is, and why.